

Modelagem Geométrica

Modelagem geométrica na Computação Gráfica

Computação gráfica em tempo real gera imagens interativas com resposta imediata ao usuário

- Exemplos: jogos, simuladores e visualizações interativas

Antes de aparecer na tela, todo objeto precisa ter sua forma descrita matematicamente

O computador não entende objetos ou superfícies - apenas **dados numéricos**

Modelagem geométrica é o processo de **descrever a forma dos objetos** usando **estruturas matemáticas**

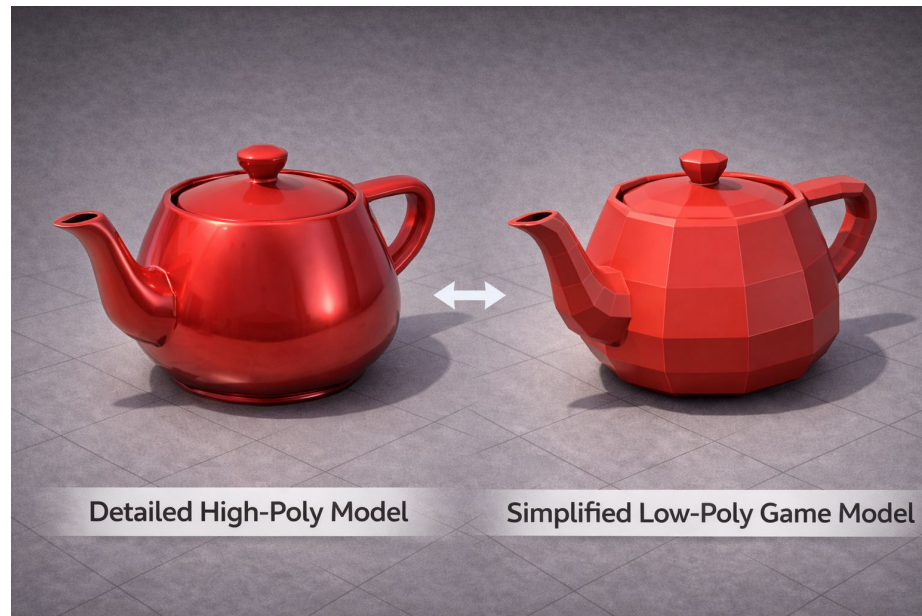
Modelagem geométrica em tempo real

No contexto de tempo real, a modelagem geométrica tem restrições claras.

Características principais:

- Precisa ser simples
- Precisa ser eficiente
- Precisa ser previsível em custo computacional

A forma não precisa ser perfeita, apenas boa o suficiente para parecer correta em movimento.



Representações geométricas

Existem diferentes formas de representar um objeto geometricamente

Duas abordagens principais:

- **Representação implícita**

- Forma definida por uma função matemática
- Exemplo: equação de uma esfera
- Pontos que satisfazem a equação pertencem à forma

- **Representação explícita**

- Forma descrita diretamente por pontos no espaço
- Define vértices e conexões entre eles

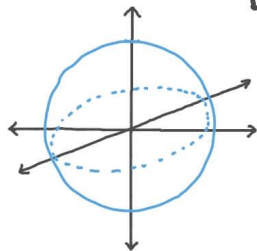
Exemplo - Equação da Esfera

$$(x-\underline{a})^2 + (y-\underline{b})^2 + (z-\underline{c})^2 = \underline{r}^2 \quad \leftarrow \text{Standard Form}$$

Center: (a, b, c)

radius: r

EQUATION OF A SPHERE



$$(x-\underline{1})^2 + (y-\underline{-3})^2 + \underline{z}^2 = \underline{9}$$

Center: $(1, -3, 0)$

radius: 3

Representação explícita vs implícita

Representação implícita

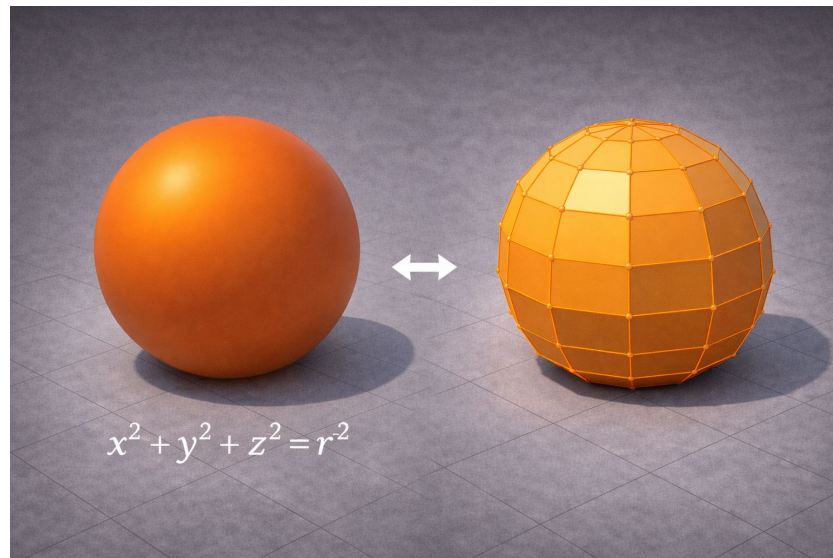
- Forma definida por **equações matemáticas**

Representação explícita

- Forma descrita diretamente por **vértices e conexões**

Na computação gráfica em tempo real:

- representação explícita é preferida
- fácil de armazenar em memória
- funciona naturalmente com a GPU



Representações geométricas possíveis

A forma de um objeto pode ser representada de diferentes maneiras.

As principais categorias são:

- malhas poligonais
- curvas e superfícies paramétricas
- representações implícitas
- representações volumétricas

Todas descrevem geometria, mas com custos e usos diferentes.

Outras formas de representação geométrica

Além das malhas poligonais, existem outras representações:

- **Curvas e superfícies paramétricas**
 - Bézier
 - B-splines
 - NURBS
 - comuns em CAD e design industrial
- **Representações implícitas e volumétricas**
 - campos escalares
 - isosuperfícies
 - voxels

Essas representações normalmente precisam ser **convertidas para triângulos** para renderização em tempo real.

Malhas poligonais

No tempo real, toda geometria precisa ser **convertida para triângulos**

A malha poligonal é o **formato final** usado pela GPU

Uma malha poligonal é:

- conjunto de polígonos **conectados**
- aproximação da **superfície** de um objeto
- estrutura compatível com **hardware gráfico**

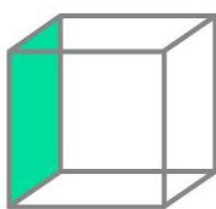
Por isso, praticamente todos os jogos usam malhas poligonais.



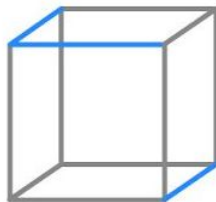
Elementos de uma malha

Uma malha é construída a partir de três elementos básicos:

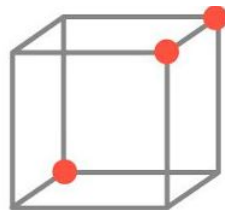
- **Vértices**
 - pontos no espaço definidos por coordenadas (x, y, z)
- **Arestas**
 - conexões entre dois vértices
- **Faces**
 - superfícies formadas pela conexão de vários vértices
 - recebem iluminação e textura



FACE



ARESTA



VÉRTICE

Forma geométrica vs estrutura topológica

Uma malha não descreve apenas forma, mas também estrutura.

A topologia define:

- como vértices, arestas e faces se conectam
- a continuidade da superfície
- a orientação das faces

Duas malhas podem ter a mesma forma, mas topologias diferentes.

Malhas manifold e não-manifold

Uma malha bem formada segue regras topológicas.

Exemplo comum:

- cada aresta pertence a no máximo duas faces

Malhas que violam essas regras podem causar:

- erros de iluminação
- problemas de visibilidade
- falhas em sombras e colisões

Triângulos como primitiva da GPU

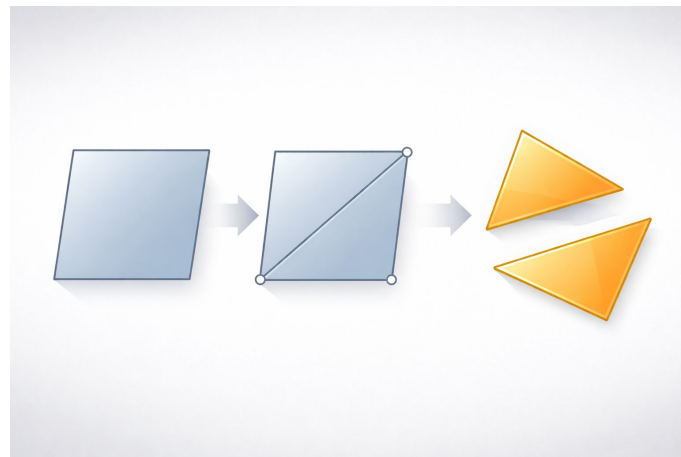
Uma face pode ter diferentes formatos:

- triângulos
- quadriláteros
- polígonos complexos

Porém, a GPU trabalha fundamentalmente com **triângulos** porque:

- **três pontos** sempre definem um **plano**
- **não** há **ambiguidade geométrica**
- processamento é simples e eficiente

Por isso, toda geometria é convertida em triângulos na renderização.



O que é triangularização

Triangularização é o processo de dividir superfícies em triângulos.

Esse processo:

- Acontece automaticamente em muitos casos
- Pode ser feito de diferentes maneiras
- Afeta diretamente a aparência do objeto

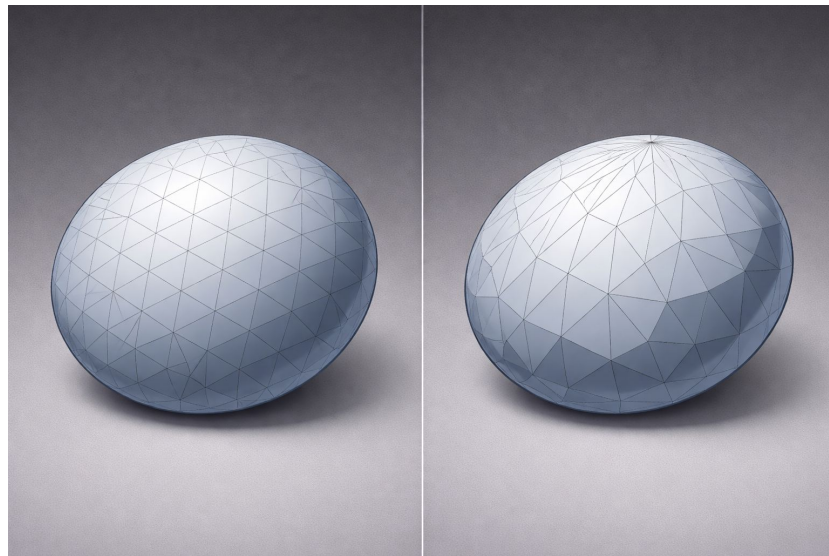
A mesma forma pode gerar malhas muito diferentes dependendo da triangularização.

Impacto visual da triangularização

A forma como os triângulos são distribuídos influencia:

- Suavidade visual
- Aparência de curvas
- Presença de artefatos visuais

Triângulos mal distribuídos podem causar distorções mesmo em objetos simples.



Normais geométricas

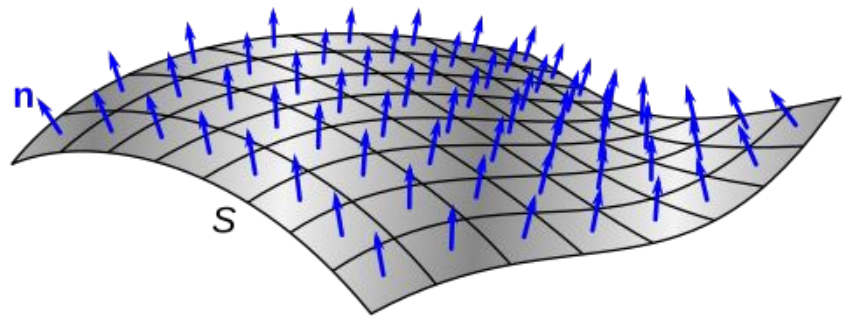
Além da posição, a geometria precisa indicar orientação.

Isso é feito por vetores normais.

A normal define:

- para que lado a superfície está voltada
- como a luz interage com a superfície

Sem normais, não existe iluminação correta.



Normal de face e normal de vértice

Normal de face:

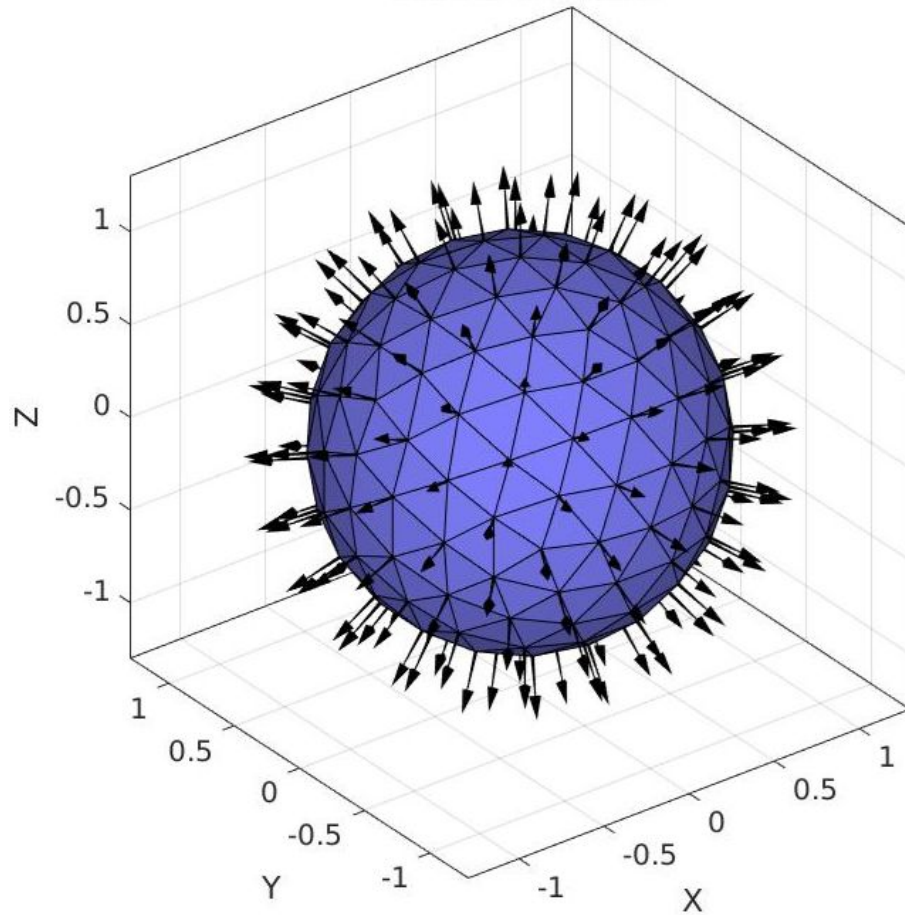
- única para todo o triângulo
- aparência facetada

Normal de vértice:

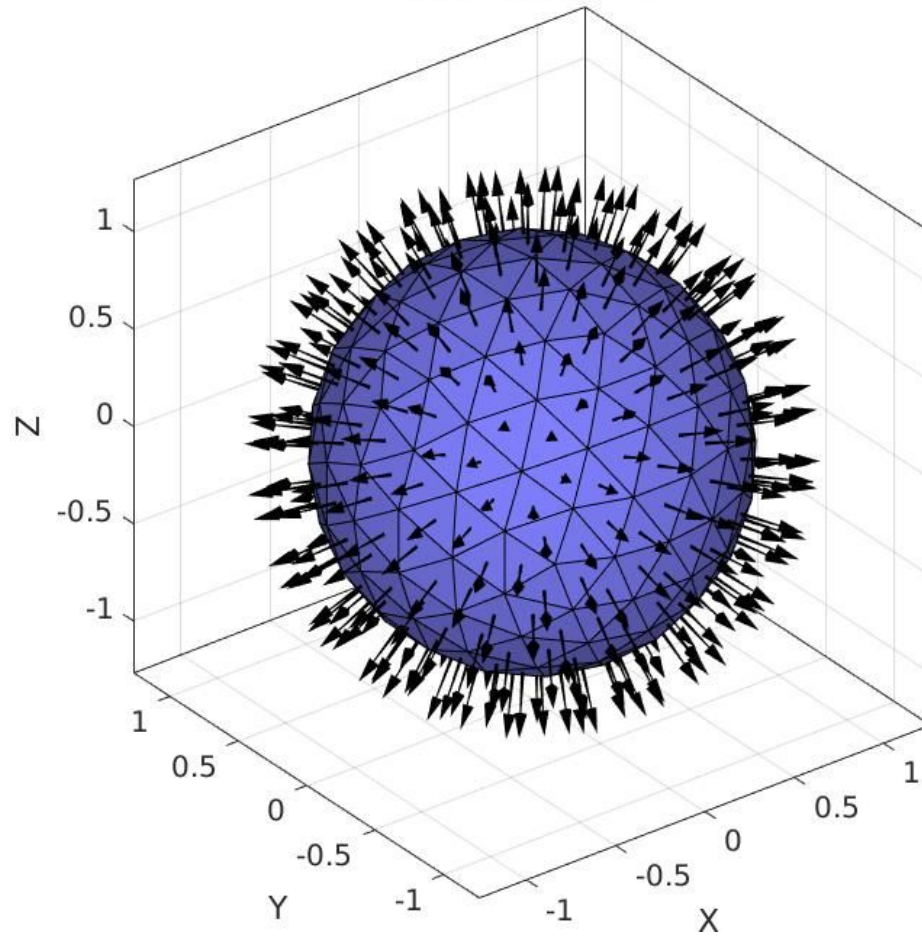
- média das faces adjacentes
- aparência suave

A escolha afeta diretamente o resultado visual.

Vertex normals



Face normals



Custo geométrico

O desempenho em tempo real depende diretamente da **quantidade de triângulos**.

Relação geral:

- poucos triângulos → menor custo computacional
- muitos triângulos → maior detalhe visual

O custo final depende:

- da soma de todos os triângulos da cena
- da complexidade visual desejada
- do hardware disponível

Nível de detalhe geométrico

Nem todo objeto precisa do mesmo nível de detalhe o tempo todo.

Level of Detail (LOD) é o uso de múltiplas versões geométricas do mesmo objeto.

Cada versão tem:

- quantidade diferente de triângulos
- custo computacional diferente



Por que LOD é necessário

Objetos distantes:

- ocupam poucos pixels
- não precisam de alta complexidade geométrica

Sem LOD:

- desperdício de processamento
- queda de desempenho

LOD é uma técnica geométrica, não apenas visual.

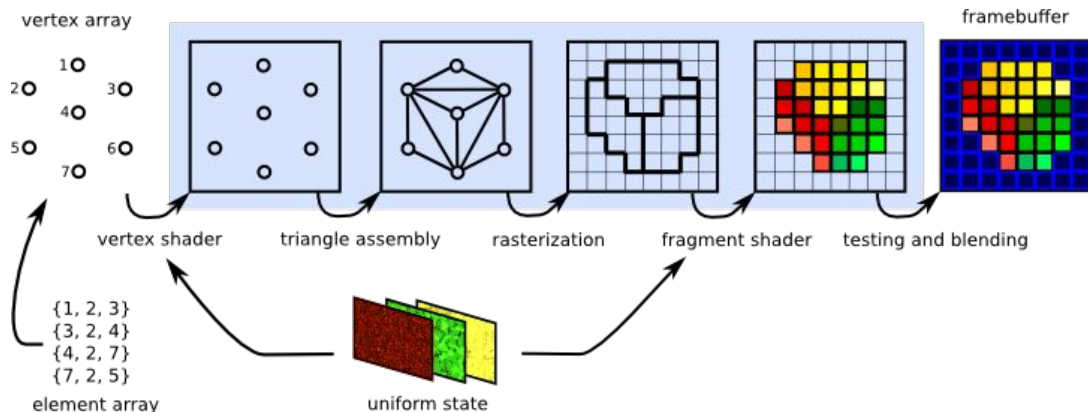
Geometria como dados

Na computação gráfica em tempo real, objetos são representados como **dados estruturados**.

Esses dados descrevem:

- posições dos vértices
- conexões entre vértices
- quais conjuntos de vértices formam triângulos

A GPU interpreta esses dados para gerar a imagem.



Vértices e atributos

Cada vértice pode conter dados associados:

- posição (x, y, z)
- normal
- coordenadas de textura
- outros atributos

Uma malha é uma coleção organizada desses vértices.

Índices e montagem da geometria

Triângulos são formados **combinando vértices existentes**.

Cada triângulo usa:

- três vértices

Para evitar duplicação de dados:

- usamos **índices que apontam para vértices já existentes**
- isso reduz memória e melhora desempenho

Por que isso importa no tempo real

A forma como a geometria é organizada afeta diretamente:

- desempenho
- uso de memória
- escalabilidade da cena

Em aplicações em tempo real, essas decisões impactam a taxa de quadros e a fluidez da experiência.

Geometria auxiliar

A malha completa raramente é usada para testes diretos. Em vez disso, usamos aproximações geométricas simples.

Essas aproximações são chamadas de **bounding volumes**.

Exemplos frequentes:

- caixas alinhadas aos eixos (AABB)
- caixas orientadas (OBB)
- esferas envolventes

Elas não representam a forma exata, apenas o volume ocupado.

Para que bounding volumes são usadas

Bounding volumes são usadas para:

- testes de visibilidade
- culling
- colisão
- seleção de objetos

Elas conectam modelagem geométrica com desempenho.

Conclusão

Na computação gráfica em tempo real:

- **geometria** é representada como **dados estruturados**
- **triângulos** são a **base da renderização**
- **mais detalhe** visual implica maior **custo computacional**

Modelagem geométrica é sempre um **equilíbrio** entre:

- qualidade visual
- desempenho
- simplicidade da geometria